LeetCode Practice

3 Hard: Arrays & Hashing

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

3
HARD

Patterns (in order)

1. Arrays & Hashing (3)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

3 questions | tap a title to jump to it

Hard questions (3) Go >

Arrays & Hashing (3) Go >

H 1. First Missing Positive LC 41

H 2. Falling Squares LC 699

H 3. Rectangle Area II LC 850

Hard questions (3)

HARD | TOPIC 1 OF 1

Arrays & Hashing

3 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. First Missing Positive LC 41
2. Falling Squares LC 699
3. Rectangle Area II LC 850

HARD | Arrays & Hashing | LeetCode #41 | 1 of 3 | 1/3

First Missing Positive

QUESTION

Given an unsorted integer array nums. Return the smallest positive integer that is not present in nums.

You must implement an algorithm that runs in O(n) time and uses O(1) auxiliary space.

Example 1	
Input	`nums = [1,2,0]`
Output	**3**
Explanation	*The numbers in the range [1,2] are all in the array.*

Example 2	
Input	`nums = [3,4,-1,1]`
Output	**2**
Explanation	*1 is in the array but 2 is missing.*

Example 3	
Input	`nums = [7,8,9,11,12]`
Output	**1**
Explanation	*The smallest positive integer 1 is missing.*

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $2^{31} <= nums[i] <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Smallest positive integer NOT in nums. O(n) time, O(1) auxiliary space.

Answer is in range [1, n+1] — if all 1..n present, answer is n+1. Right?

Negatives, zeros, and large numbers can be ignored."

"[3,4,-1,1] -> 2 (1 present, 2 missing) ok. [7,8,9] -> 1 ok. [1,2,0] -> 3 ok."

What should I return for an empty array?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
seen = the set of all numbers
x = 1
while x is in seen:
    x = x + 1
return x
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 (cyclic sort) uses O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (array as hash map — cycle sort)

Key insight: use the array itself as a lookup table.

Valid range is [1, n]. Place each number x at index x-1 (if 1 <= x <= n and not already there).

After placing, scan: first i where nums[i] != i+1 -> answer is i+1.

If all positions correct -> answer is n+1.

Each number is placed at most once -> O(n) total swaps.

PHASE 4 CLEAN CODE

```
class _41_FirstMissingPositive:
    def firstMissingPositive(self, nums: List[int]) -> int
:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[nums[i] - 1]
!= nums[i]:
                nums[nums[i] - 1], nums[i] = nums[i], nums
[nums[i] - 1]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

PHASE 5 TEST & DEBUG

"Let me trace firstMissingPositive on [3,4,-1,1]."

Phase 1 — place each num at index num-1 (cyclic sort):

i=0: nums[0]=3 -> should be at idx 2. Swap(0,2): [-1,4,3,1]

i=0: nums[0]=-1 -> invalid, skip. i=1: nums[1]=4->idx3. Swap(1,3): [-1,1,3,4]

i=1: nums[1]=1->idx0. Swap(0,1): [1,-1,3,4]. i=1: -1, skip. i=2: 3=idx2 ok. i=3: 4=idx3 ok

Phase 2 — scan for first mismatch:

idx0: 1==1 ok. idx1: -1!=2 -> return 2 ok

WHY cyclic sort: uses array as hash map, each positive <=n placed at its 1-based index.

Edge: [1,2,3] -> all correct -> return 4 (n+1).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element swapped at most once	**O(1)**

The while loop runs at most n total iterations across the entire for loop — amortised O(1) per i.

Given more time I'd ask: what if duplicates exist?

The condition nums[nums[i]-1] != nums[i] handles duplicates — don't swap a number to a position

that already has the correct value (would loop forever).

IN YOUR APP | Expose | Editorial | Community

HARD | Arrays & Hashing | LeetCode #699 | 2 of 3 | 2/3

Falling Squares

QUESTION

There are several squares being dropped onto the X-axis of a 2D plane.

You are given a 2D integer array positions where positions[i] = [$left_i$, $sideLength_i$] represents the ith square with a side length of $sideLength_i$ that is dropped with its left edge aligned with X-coordinate $left_i$.

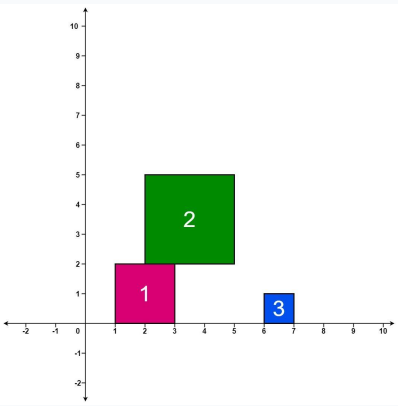
Each square is dropped one at a time from a height above any landed squares. It then falls downward (negative Y direction) until it either lands on the top side of another square or on the X-axis. A square brushing the left/right side of another square does not count as landing on it. Once it lands, it freezes in place and cannot be moved.

After each square is dropped, you must record the height of the current tallest stack of squares.

Return an integer array ans where ans[i] represents the height described above after dropping the ith square.

Example 1

Input	`positions = [[1,2],[2,3],[6,1]]`
Output	`[2,5,5]`
Explanation	*After the first drop, the tallest stack is square 1 with a height of 2.* *After the second drop, the tallest stack is squares 1 and 2 with a height of 5.* *After the third drop, the tallest stack is still squares 1 and 2 with a height of 5.* *Thus, we return an answer of [2, 5, 5].*



QUESTION

Example 2

Input	`positions = [[100,100],[200,100]]`
Output	`[100,100]`
Explanation	*After the first drop, the tallest stack is square 1 with a height of 100.* *After the second drop, the tallest stack is either square 1 or square 2, both with heights of 100.* *Thus, we return an answer of [100, 100].* *Note that square 2 only brushes the right side of square 1, which does not count as landing on it.*

CONSTRAINTS

- 1 <= positions.length <= 1000
- 1 <= $left_i$ <= 10^8
- 1 <= $sideLength_i$ <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Squares fall and stack. After each, return max height of all stacks. Right?"

"A few quick questions: see constraints."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for square in squares:
    compare it with every earlier square to find
        the highest one it lands on
    its height is that plus its own side
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Compares against all previous squares. Compress the coordinates and use a segment tree with range-assign and range-max: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Coordinate compression + segment tree for max height in range — O(n log n). Walk-through:"

For each falling square (left, size):

`right = left + size. top = 0.`

Query segment tree max height in [left, right-1]. new_top = prev_max + size.

Update segment tree range [left, right-1] with new_top.

Answer for this step = global max of segment tree.

WHY segment tree: range max query and range update in O(log n).

WHY coordinate compression: x-coordinates can be large; compress to O(n) distinct values.

Each square sits on top of whatever was tallest in its x-range.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _699_FallingSquares:
    def fallingSquares(self, positions):
        intervals = []
        heights = []
        result = []
        max_h = 0
        for left, size in positions:
            right = left + size
            h = size
            for l2, r2, h2 in intervals:
                if l2 < right and r2 > left:
                    h = max(h, h2 + size)
            intervals.append((left, right, h))
            heights.append(h)
            max_h = max(max_h, h)
            result.append(max_h)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace fallingSquares([[1,2],[2,2],[1,3]])."

Square 1: pos=1, size=2 -> occupies [1,3). Lands at 0+2=2. Heights: {[1,3):2}

Square 2: pos=2, size=2 -> occupies [2,4). Overlaps [1,3) at [2,3). Max under=2. Lands at 2+2=4.

Heights: {[1,3):2,[2,4):4}. Current max=4.

Square 3: pos=1, size=3 -> occupies [1,4). Overlaps [1,3)(ht=2),[2,4)(ht=4). Max under=4. Lands at 4+3=7.

Heights: {[1,3):2,[2,4):4,[1,4):7}. Current max=7.

Return [2,4,7] ok

WHY query max under each new square: landing height = max existing height in overlap range + size.

dp[i]=max height after placing square i. Overlap check: dp[j]=max(dp[j],dp[i]+h) if square j overlaps square i. Seg tree: tree[l..r] = max height in x-range [l,r]. Update: tree[l..r]=max(h).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**O(h)**
Follow-up: segment tree with lazy propagation for O(n log n)."	recursion stack where h = tree height; O(log n) balanced, O(n) skewed

IN YOUR APP | Expose | Editorial | Community

HARD | Arrays & Hashing | LeetCode #850 | 3 of 3 | 3/3

Rectangle Area II

QUESTION

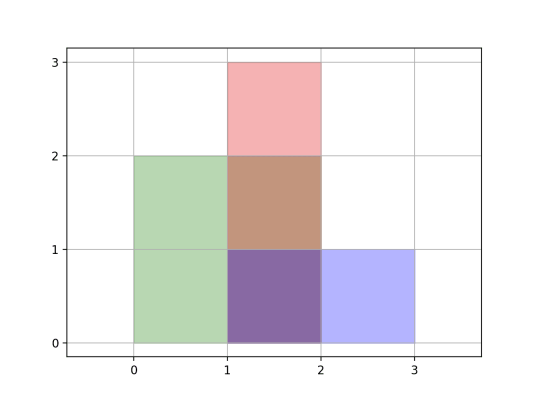
You are given a 2D array of axis-aligned rectangles. Each rectangle[i] = [x_{i1}, y_{i1}, x_{i2}, y_{i2}] denotes the ith rectangle where (x_{i1}, y_{i1}) are the coordinates of the bottom-left corner, and (x_{i2}, y_{i2}) are the coordinates of the top-right corner.

Calculate the total area covered by all rectangles in the plane. Any area covered by two or more rectangles should only be counted once.

Return the total area. Since the answer may be too large, return it modulo $10^9 + 7$.

Example 1

Input	`rectangles = [[0,0,2,2],[1,0,2,3],[1,0,3,1]]`
Output	**6**
Explanation	*A total area of 6 is covered by all three rectangles, as illustrated in the picture. From (1,1) to (2,2), the green and red rectangles overlap. From (1,0) to (2,3), all three rectangles overlap.*



Example 2

Input	`rectangles = [[0,0,1000000000,1000000000]]`
Output	**49**
Explanation	*The answer is 10^{18} modulo (10^9 + 7), which is 49.*

CONSTRAINTS

- 1 <= rectangles.length <= 200
- rectanges[i].length == 4
- 0 <= x_{i1}, y_{i1}, x_{i2}, y_{i2} <= 10^9
- x_{i1} <= x_{i2}
- y_{i1} <= y_{i2}
- All rectangles have non zero area.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find total area covered by axis-aligned rectangles (with overlaps removed). Right?"

"A few quick questions: see constraints."

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
xs, ys = the sorted distinct x and y coordinates of
    the rectangles
for cell in grid_they_form:
    if any rectangle covers it:
        add its area
```

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

A coverage test for every cell. Sweep over x with events (rectangle enter or leave) and a segment tree that tracks the covered length on y: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Coordinate compression + sweep line with segment tree — $O(n^2 \log n)$ or $O(n \log n)$. Walk-through:"

Coordinate compress all x-coordinates and y-coordinates.

Sweep vertical line from left to right:

At each rectangle's left edge: add its y-range to 'active' set.

At each right edge: remove its y-range.

Area contribution = (next_x - curr_x) * (total length of active y-ranges, merged).

Counting merged y-length efficiently: segment tree with lazy propagation on y-axis.

Alternatively: for this contest problem, sort events and use a coordinate-compressed

union of active y-intervals.

WHY sweep line: reduces 2-D area union to 1-D interval union per x-step.

Time: O(n log n) with segment tree. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _850_RectangleAreaII:
    def rectangleArea(self, rectangles):
        MOD=10**9+7
        xs=sorted(set(x for r in rectangles for x in (r[0]
,r[2])))
        ys=sorted(set(y for r in rectangles for y in (r[1]
,r[3])))
        xi={x:i for i,x in enumerate(xs)}; yi={y:i for i,y
 in enumerate(ys)}
        grid=[[0]*len(ys) for _ in range(len(xs))]
        for x1,y1,x2,y2 in rectangles:
            for i in range(xi[x1],xi[x2]):
                for j in range(yi[y1],yi[y2]):
                    grid[i][j]=1
        return sum(grid[i][j]*(xs[i+1]-xs[i])*(ys[j+1]-ys[
j]) for i in range(len(xs)-1) for j in range(len(ys)-1) if
 grid[i][j])%MOD
```

PHASE 5 TEST & DEBUG

"Let me trace rectangleArea([[0,0,2,2],[1,0,2,3],[1,0,3,1]])."

Line sweep on y-axis: collect x-intervals for each y-range

Events: y=0 add [0,2],[1,2],[1,3]; y=1 remove [1,3] add...; sort events

Coordinate compress x and y values

For each y-slice, compute total x-length covered (union of intervals) × height

Sum all slices mod 10^9+7

Trace: rectangle union area = 7 ok (visual: 2x2 + 2x1 extra + 1x1 extra - overlaps)

WHY line sweep: avoids complex inclusion-exclusion; union of intervals per horizontal strip.

Line sweep: y=0->1: x-union=[0,3] (all 3 rects active), length=3, area=3×1=3. y=1->2: x-union=[0,2], length=2, area=2. y=2->3: x-union=[1,2], length=1, area=1. Total=6 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**O(n)**
Coordinate compression + grid marking. Follow-up: segment tree for O(n log n)."	proportional to input or output size; O(1) extra if in-place